Title for Report

Subtitle for Report

Authors: Aymen Tiguite, Cindy Frempong

Boston University | DS 340

April Due Date 2026

Include at least 3 figures or tables to illustrate your data, methods, training, and/or results.
Format: Times new roman 12, single space, sections are bolded

Introduction: What is the problem you were trying to solve and why did you want to solve it? Also include any background information necessary to understand the problem (such as explaining sports statistics used later in the paper) (context)

Music genre classification is a classic problem in audio and music information retrieval (MIR). Automatically identifying a song's genre has real world applications across the music industry, from powering recommendation engines on streaming platforms like Spotify and Apple Music, to organizing large digital music libraries, to enabling content-based search and music discovery. Despite its practical importance, genre classification is a genuinely hard problem: genre boundaries are fuzzy, listener perceptions vary, and many songs blend elements of multiple genres simultaneously.

In this paper, we explore multiple model types and ensemble methods to eventually train a convolutional neural network(CNN) in correctly identify the genre of a given song. We draw on a dataset of tracks described by a set of numerical audio descriptors that summarize musical properties, from Spotify's audio analysis API. The features summarize the feel and energy of a song, sound source, and musical structure of a song. For example, danceability estimates how suitable a track is for dancing, energy reflects intensity and activity, valence measures how positive or happy a track sounds, and acousticness estimates whether a song is more acoustic versus electronically produced.

In addition to these descriptors, we make use of Mel-Frequency Cepstral Coefficients (MFCCs). MFCCs are a compact numerical representation of the short-term power spectrum of a sound, computed by mapping the audio signal onto the mel scale, a perceptual scale of pitch that closely mirrors how human hearing works. MFCCs are widely used in speech and music processing because they capture the texture of a sound in a way that correlates well with human perception of musical style. In our work, MFCCs serve as a primary input representation, allowing the CNN to learn patterns in the spectral content of songs that are characteristic of different genres.

Data Description:

Data Source:

Source #1: Spotify Tracks Dataset

https://www.kaggle.com/datasets/yashdev01/spotify-tracks-dataset

The dataset includes information on Spotify tracks spanning 125 different music genres, with each row representing an individual song and each column capturing a specific attribute of that track. 80,000 rows and 22 columns

Source #2: MFCC Dataset of Spotify tracks

This dataset can be found on our GitHub repo under data(mfccs_features.csv). We manually extracted the MFCC values for a sample of tracks from the Spotify dataset. For each track, we compute 13 MFCC coefficients and summarize them using three statistical descriptors: the mean (average value of each coefficient across the track), the standard deviation (how much each coefficient varies over time), and the delta (the rate of change of each coefficient, capturing how the spectral texture evolves throughout the song). This produces 39 features per track, alongside the track_id and track_genre.

EDA and Dataset Overview:

Data Preprocessing and Cleaning (Data Quality):

Spotify Tracks Dataset: We started off with relatively clean data, we checked for nulls there was only 3 nulls, dropped those, identify columns present and data types for those columns,
Before we input our data for our baseline models we encoded genre names into numerical

Exploratory Data Analysis

We began our exploratory data analysis by examining the distribution of genres across the dataset. The dataset contains 114 unique genres, each represented by exactly 1,000 tracks, resulting in a perfectly balanced class distribution with each genre accounting for approximately 0.88% of the total data. This balance is ideal for training our various baseline models, as it ensures they are not biased toward any particular genre during learning.

track_genre	count	proportion
acoustic	1000	0.0088
punk-rock	1000	0.0088
progressive-house	1000	0.0088
power-pop	1000	0.0088
pop	1000	0.0088
pop-film	1000	0.0088
piano	1000	0.0088
party	1000	0.0088
pagode	1000	0.0088
opera	1000	0.0088
new-age	1000	0.0088

We next investigated whether individual tracks appeared under multiple genre labels in the dataset. We found that 16,299 tracks are assigned to more than one genre(and sub-genres), revealing significant overlap in how songs are categorized, reflecting the reality that many songs genuinely blend elements of multiple genres. This raises an important modeling consideration we explore in our models: a prediction is counted as correct if it matches any one of a track's valid genres, rather than requiring an exact match to a single genre.

track_id	track_name	artists	track_genre
001APMDOl3qtx1526T11n1	Better	Pink Sweat$;Kirby	soul
001APMDOl3qtx1526T11n1	Better	Pink Sweat$;Kirby	chill
001YQlnDSduXd5LgBd66gT	El Tiempo Es Dinero - Remasterizado 2007	Soda Stereo	punk-rock
001YQlnDSduXd5LgBd66gT	El Tiempo Es Dinero - Remasterizado 2007	Soda Stereo	ska
003vvx7Niy0yvhvHt4a68B	Mr. Brightside	The Killers	rock
...	...	...	...
7zv2vmZq8OjS54BxFzI2wM	Lets Start the Party	Attila	metalcore
7zv2vmZq8OjS54BxFzI2wM	Lets Start the Party	Attila	death-metal
7zwn1eykZtZ5LODrf7c0tS	You Get Me So High	The Neighbourhood	alternative
7zwn1eykZtZ5LODrf7c0tS	You Get Me So High	The Neighbourhood	rock
7zwn1eykZtZ5LODrf7c0tS	You Get Me So High	The Neighbourhood	alt-rock

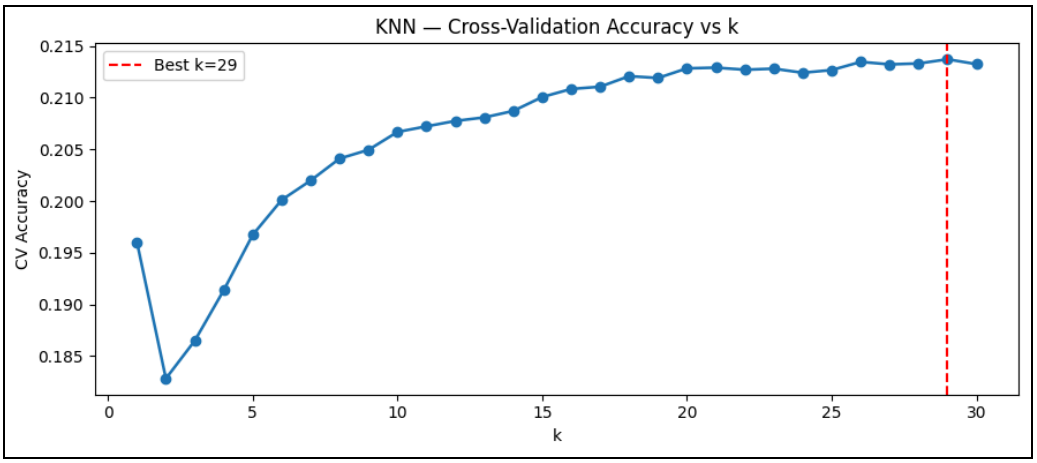
Methodology-What did you do? Include details like which parameters you tried.

Baseline Models

We trained a set of baseline models to establish a performance benchmark that our final model would need to surpass.

K-Nearest Neighbors

A KNN classifier trained on the 15 Spotify audio features. Before training, we standardized all features using a standard scaler and applied PCA to reduce dimensionality while retaining 95% of the variance. We tuned the number of neighbors k using 5-fold cross-validation across values from 1 to 30, selecting the value of k that maximized average validation accuracy. The KNN baseline achieved only 21% accuracy on the test set. KNN struggles in high-dimensional spaces, and the raw audio descriptors alone do not seem sufficient to reliably distinguish between 114 genres.



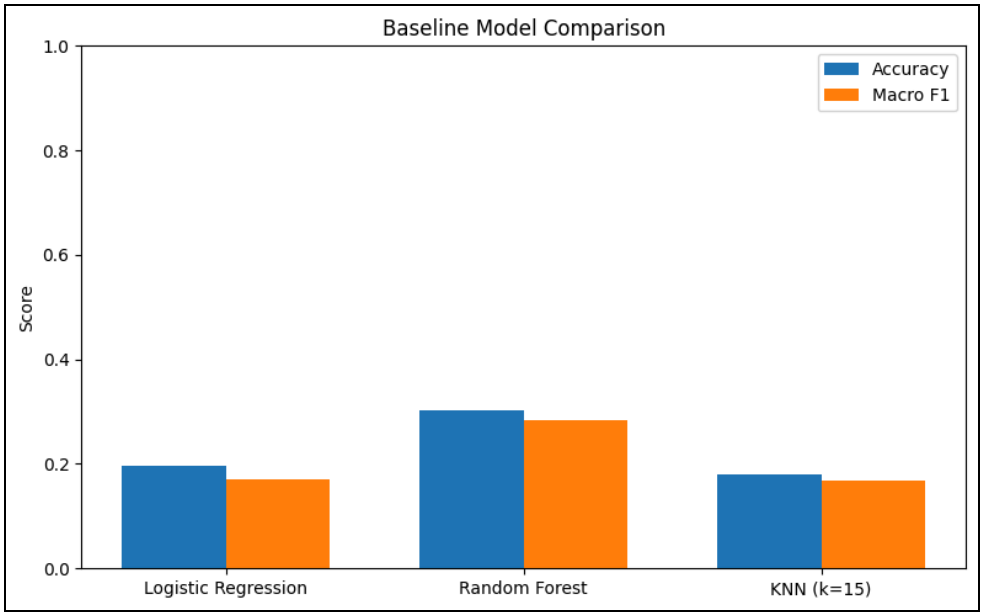
We then grouped the 114 genres into 12 broader bins and retrained the KNN classifier to see if simplifying the task would improve performance. Accuracy nearly doubled from 21% to 44%, with electronic and ambient/instrumental being the easiest to classify and hip-hop/r&b and indie/emo the hardest.

Logistic Regression

A Logistic Regression classifier trained on the same 15 Spotify audio features. Due to memory constraints in our development environment, we worked with a stratified sample of 300 tracks per genre, and split the data 70/15/15 into train, validation, and test sets. Features were standardized using a standard scaler fit on the training set only, and the model was trained using the saga solver to handle the large number of classes efficiently. The model achieved a validation accuracy of 19.7% and a macro F1 score of 0.17 across 114 genres

Random Forest

A Random Forest classifier consisting of 100 decision trees with a maximum depth of 15, trained on the same setup as the previous baselines, a sample of 300 tracks per genre, a 70/15/15 train/validation/test split, and standardized features. The model achieved a validation accuracy of 30.2% and a macro F1 of 0.28



Ensemble (XGBoost and LGB)

Similar to the knn, the model was trained on the binned version of the data. We trained it with 200 estimators, a maximum depth of 6, and a learning rate of 0.1 on the same stratified sample and 70/15/15 split as the previous baselines. XGBoost achieved a validation accuracy of 50.3% and a macro F1 of 0.47, the strongest performance of any baseline model.

After evaluating our baseline models, we determined that classifying across all 114 genres was too granular a task for the audio features available to us. We therefore adopted a genre binning approach, consolidating the original 114 genres into 12 broader categories for most future models. Although we have some intermediate models that were also trained on the full unbinned label set for experimentation and comparison purposes, our primary goal is to build a model that accurately classifies songs across these 12 binned genres.

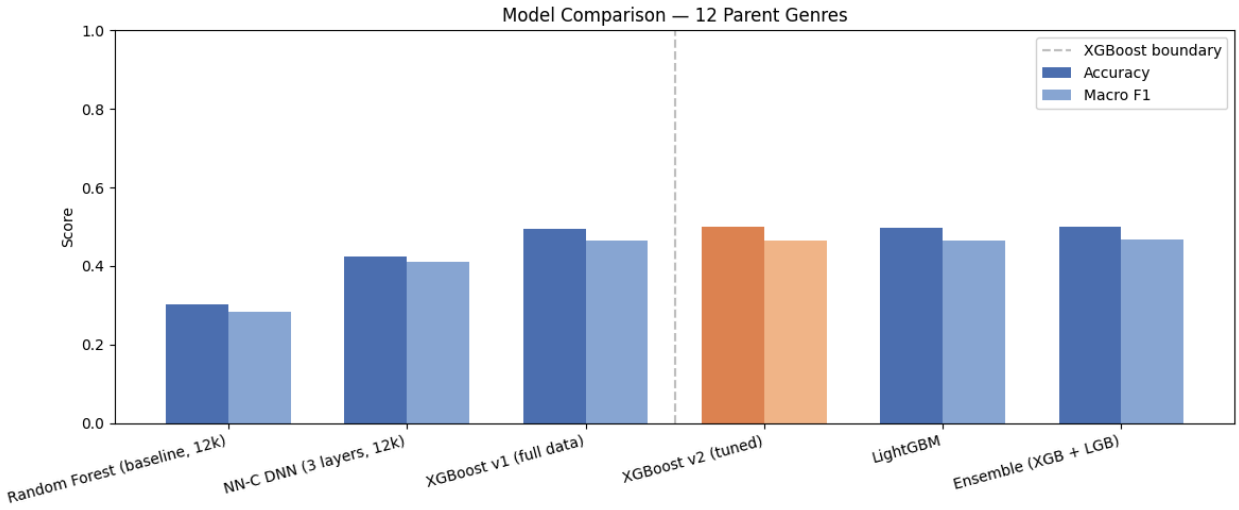
Intermediate Models (aymen)

Model A: Model description, model training process, model results/ output

Model B: Model description, model training process, model results/ output

Model C: Model description, model training process, model results/ output

Feature Engineering: (aymen)



MFCC Dataset Extraction

After evaluating our intermediate models, we found that the Spotify audio features alone were not providing enough information for the model to reliably distinguish between genres. We decided to incorporate our MFCCs, hoping the added spectral detail would meaningfully boost performance. Before building our own MFCC dataset, we first searched for a pre-existing one to save time. However, the datasets we found contained very outdated songs, and some tracks were even misclassified.

To ensure a balanced dataset for MFCC extraction, a hybrid sampling strategy was applied across genres. A minimum floor of 50 samples per genre was enforced, with 10 samples drawn per subgenre where possible. For genres with fewer subgenres, such as Reggae and Jazz/Blues, the per-subgenre allocation was scaled up proportionally to meet the 50-sample minimum. This ensures no genre is underrepresented during classifier training while still preserving subgenre diversity within each genre.

To build our MFCC dataset, we iterated a custom extraction pipeline that automated the process of retrieving audio and computing features for each track. For each song in the dataset, the pipeline searched YouTube using the track name and artist, downloaded a 30 second audio clip centered around the midpoint of the song using the yt-dlp library, and extracted 13 MFCC coefficients using librosa. Each track was summarized by three statistics per coefficient the mean, standard deviation, and delta, producing a 39 dimensional feature vector per track. Tracks

were skipped and flagged if audio could not be retrieved from YouTube or if MFCC extraction failed.

We acknowledge that our YouTube extraction method is not perfect since search results could occasionally return live versions, covers, or the wrong song entirely. We mitigated this by including "official audio" in the search query, though some mismatches might still remain in the dataset. The output of the extraction was validated by.

Finalized Models

This next iteration of models utilize the MFCC dataset and intermedite models in ensemble methods to…

Final combo:

(XGB(parent) + CNN with mfccs (parent) → + LGB(sub genere for metal)

Evaluation Metrics and Results

[image of table]

Results - What were your results? However you evaluated your system, put the results here. Include the results of parameter search as well as results of the experiments you proposed in your original proposal.

Conclusion

- Impact and Real World Application

- Limitations and Future Work
- **Conclusions - What did you conclude from your results? Is there anything else your team learned from the experience?**
-
-

Baseline for random chance -> include somewhere in report
.0087 chance of getting genre correct 1/ 114 the chances of getting a parent genre correct 1/12, is .083(methodology)

AI Usage Disclaimer